



Coroutines

En Android, muchas tareas requieren ejecutarse de forma asíncrona para no bloquear el hilo principal (UI thread). Las corutinas permiten hacer esto de manera sencilla: podemos ejecutar operaciones simultáneas sin que una tarea interfiera o congele a otra, manteniendo la app fluida y responsiva.

En otras palabras, las corutinas son una herramienta de Kotlin para manejar operaciones concurrentes o paralelas de forma segura y eficiente, evitando el clásico "callback hell" y la saturación del hilo principal.

~~Para poder utilizar y ejecutar las corrutinas de forma genérica kotlin nos ofrece tres posibilidades de ejecución "por defecto"~~

- ~~• Global Scope: utilizado para cualquier corrutina que debe continuar su ejecución mientras la aplicación esté activa. Esto quiere decir que no debe estar asociada a ningún elemento de la aplicación, como por ejemplo fragments. Para poder utilizarla es necesario lanzarla de la siguiente forma—~~

```
GlobalScope.launch(Dispatchers.IO){  
    // acción a ejecutar en 2º plano  
}
```

lifecycleScope es una extensión de LifecycleOwner (como Activity o Fragment) que te permite lanzar coroutines que se cancelan automáticamente cuando la actividad o fragmento se destruye.

```
lifecycleScope.launch(Dispatchers.IO) {  
    // acción a ejecutar en segundo plano  
    val database = UsuariosBBDD.getInstance(applicationContext)  
    database?.usuarioDAO()?.insert(Usuario("Daniel",  
    "daniel.amiguet@murciaeduca.es"))  
}
```

Los tres dispatcher que tiene kotlin configurado por defecto son los siguientes:

- Main: hilo principal de la ejecución. Es el encargado de todo el tema de interfaz gráfica y configuración de los escuchadores
- IO: hilo reservado para aquellas tareas que implican un flujo de datos de entrada / salida como puede ser manejo de base de datos, comunicaciones por red, manejo de ficheros, etc...
- Default: se trata del hilo predeterminado, reservado para tareas de cálculo potentes



```
val numeros = 100000000

println("Iniciando Hilo")
lifecycleScope.launch(Dispatchers.Default) {
    var sum = 0L
    val startTime = System.currentTimeMillis()

    for (i in 0 until numeros) {
        sum += i
    }

    val endTime = System.currentTimeMillis()

    // Cambia al hilo principal (UI thread) para actualizar la
    interfaz o imprimir
    withContext(Dispatchers.Main) {
        println("Suma total: $sum")
        println("Tiempo total: ${endTime - startTime} ms")
    }
}
println("Fin metodo del Hilo")
```

```
private fun iniciarSumaParalela() {
    lifecycleScope.launch {

        val numeros = 100000000
        val numPartes = 4
        val rangoPorParte = numeros / numPartes

        println("Iniciando Coroutines (4 Hilos Paralelos)")
        val startTime = System.currentTimeMillis()
        val totalSum = coroutineScope {
            val deferredSumas: List<Deferred<Long>> = (0 until
numPartes).map { i ->
                async(Dispatchers.Default) {
                    var sum = 0L
                    val start = i * rangoPorParte
                    val end = start + rangoPorParte
                    for (j in start until end) {
                        sum += j
                    }
                    // Devolvemos la suma parcial
```



```
        return@async sum
    }
}

// Esperar todos los resultados y sumarlos
deferredSumas.awaitAll().sum()
}

val endTime = System.currentTimeMillis()

// Cambiamos al Main Dispatcher para imprimir/actualizar la
UI
withContext(Dispatchers.Main) {
    println("Suma total: $totalSum")
    println("Tiempo total: ${endTime - startTime} ms")
}
}
```